

Approximate Nearest Neighbor Search over Temporal Vector Data

Yuxiang Wang, Ziyuan He, Yongxin Tong, *Senior Member, IEEE*, Zimu Zhou, *Member, IEEE*, Zijie Ma, Yiman Zhong

Abstract—Unstructured data, such as images and texts, are increasingly represented as high-dimensional vectors for emerging AI applications like retrieval-augmented generation. A key operation in these applications is querying for vectors that are both semantically similar and temporally relevant. This operation can be formulated as Timestamp Approximate Nearest Neighbor Search (TANNS), where both the vectors and the query incorporate temporal attributes, aiming to retrieve the approximate nearest neighbors valid at the given timestamp. A naive solution is to create separate indexes for each timestamp, which enables accurate and fast searches but incurs high update latency and excessive storage demands. In this paper, we introduce the timestamp graph, a novel structure that supports rapid index updates while minimizing storage costs. Exploiting the temporal locality of changes in valid vectors, our timestamp graph effectively manages a unified index across all historical timestamps, thereby substantially reducing storage overhead. Moreover, we design the historic neighbor tree, which further compresses the space complexity to that of a single-timestamp index. In addition, we extend the Timestamp Graph and Compressed Timestamp Graph to support Interval Approximate Nearest Neighbor Search (IANNS) queries—a generalized form of TANNS queries that specify a time interval and retrieve the nearest vectors valid during that interval. Extensive evaluations on five datasets show that our method achieves over 99% accuracy while improving the query efficiency by $3.3\times$ to $138.1\times$ than existing solutions.

Index Terms—high-dimensional data, approximate nearest neighbor search

I. INTRODUCTION

Vector databases, which store and manage embeddings of diverse unstructured data (e.g., text, images, audio) as high-dimensional vectors [1], [2], have become indispensable in emerging AI workflows such as retrieval-augmented generation (RAG) [3]. Acting as external knowledge repositories, they provide AI models with contextually relevant information to mitigate hallucinations and enhance in-context learning.

This work was partially supported by National Natural Science Foundation of China (NSFC) (Nos. 62425202, 62336003), Beijing Natural Science Foundation (Z230001), Fundamental Research Funds for the Central Universities (No. JKF-2026007844235), and the State Key Laboratory of Complex & Critical Software Environment (SKLCCSE). Z. Zhou’s research is supported by NSFC (No. 62572412) and City University of Hong Kong Shenzhen Research Institute (CityUHKSR).

Y. Wang, Z. He, Y. Tong, Z. Ma and Y. Zhong are with the State Key Laboratory of Software Development Environment and Beijing Advanced Innovation Center for Future Blockchain and Privacy Computing, School of Computer Science, Beihang University, China. E-mail: {yuxiangwang, hzy_he, yxtong, mazj, yeeman}@buaa.edu.cn.

Z. Zhou is with City University of Hong Kong Shenzhen Research Institute, Shenzhen, China and Department of Data Science, City University of Hong Kong, Hong Kong, China. E-mail: zimuzhou@cityu.edu.hk.

(Corresponding authors: Yongxin Tong.)

Such information is efficiently retrieved via Approximate Nearest Neighbor Search (ANNS), an operation that returns semantically similar records for a given query vector. Beyond semantic similarity, *temporal relevance* is also critical for practical vector database applications. On the one hand, the *vectors* often have a *validity* period, as new embeddings are continuously added to the database and outdated ones expire.

For instance, in a knowledge base, the embeddings of current knowledge evolve over time [4], necessitating searching over valid data only to return accurate results. On the other hand, many *queries* are tied to a specific *timestamp* or a *time interval*, requiring answers that correspond to a particular point or period of time. This is common in applications like financial analysis and fact verification, where aligning the retrieved vectors with the queried time reference is necessary to avoid misleading results [5]. We illustrate this insight in the following example:

Example 1 (Answering Time-relevant Questions): Although large language models (LLMs) exhibit strong capabilities in question answering, they often struggle with time-relevant problems [6], such as “Q1: What was the highest closing value of the NASDAQ-100 index in Oct. 1st 2023?” and “Q2: What was the prize pool for the Counter-Strike Major Championship in 2024?” Retrieval-Augmented Generation (RAG) can enhance the accuracy of answers by supplementing the LLM with text chunks that are both relevant to the query and temporally accurate. Since the retrieval step in RAG typically uses embedding similarity to identify relevant chunks [7], it is important to consider approximate nearest neighbor search of embedded data with timestamp constraint. □

To meet these needs, we introduce *Timestamp Approximate Nearest Neighbor Search (TANNS)* and *Interval Approximate Nearest Neighbor Search (IANNS)*, two new queries that incorporate *time attributes* into both the data vectors and the query. In TANNS, given a query vector paired with a timestamp, the goal is to retrieve approximate nearest neighbors among all vectors valid at that timestamp. IANNS generalizes TANNS by allowing the query to specify a time interval instead of a single timestamp, and aims to retrieve approximate nearest neighbors from all vectors valid during that interval. Specifically, for Q1 and Q2 of Example 1, a timestamp of “Oct. 1st 2023” and a time interval of “Jan. 1st 2024 - Dec. 31st 2024” can be specified when querying the text corpus for relevant chunks. Our goal is to enable *accurate* and *efficient* TANNS and IANNS queries, as applications like chat bots demand real-time, high-quality results over millions of high-dimensional vectors [7]–[9].

TABLE I: Time and space complexity of solutions.

	Query Type	Searching Time	Updating Time	Space
Naive Graph-based Method	Timestamp	$O(M \log N)$	$O(MN \log N)$	$O(MN^2)$
	Interval		$O(MN^2 \log N)$	$O(MN^3)$
Timestamp Graph	Timestamp	$O(\log^2 N)$	$O(\log^2 N)$	$O(M^2 N)$
	Interval	$O(M^2 I \log N + \log^2 N)$		
Compressed Timestamp Graph	Timestamp	$O(\log^2 N)$	$O(\log^2 N)$	$O(MN)$
	Interval	$O(M \log^2 N)$		

(N : vector number in the dataset; M : neighbor number in the graph index; I : length of the query interval;)

Prior research fails to support efficient TANNS and IANNS queries. Early studies on nearest neighbor search with temporal constraints [10]–[12] build temporal indexes to track valid objects per timestamp for rapid search. However, they struggle for the dense vector embeddings due to the curse of dimensionality [13]. A seemingly plausible solution is the recently proposed hybrid search upon vectors associated with attributes [14]–[18]. However, they are unfit for our problems because they implicitly assume attributes with pre-determined values, whereas the expiration timestamp of a vector is typically unknown when it is added to the database.

To handle the high-dimensional vectors with nondeterministic validity, we employ graph-based indexes, which prevail in ANNS over high-dimensional data [19], [20], and build a separate index per historical timestamp of interest. While this strategy enables accurate and fast searches, it incurs notable update latency due to frequent index updates whenever the validity of any vector changes. More critically, maintaining an index for all valid vectors per timestamp consumes unacceptably high storage, with a space complexity that grows quadratically with the number of vectors.

To enhance the efficiency of TANNS queries, we propose the *timestamp graph*, a novel structure that supports rapid index updates and requires low index storage. It maintains a *single* graph index for *all* historical timestamps, thus significantly reducing storage overhead. The feasibility of this approach is underpinned by the high *temporal locality* of the changes in valid vectors, resulting in considerable overlap in their indices across time. The cost of index updates is also reduced, as only the neighbors of changed vectors need modification, which constitute a small fraction of the graph. Recognizing the redundancy in storing neighbor information, we further propose the *historic neighbor tree* to compress the timestamp graph. Remarkably, the compressed timestamp graph tracks valid vectors across all timestamps while achieving the *same space complexity* as a *single-timestamp* index.

Furthermore, we extend timestamp graph to support IANNS queries over temporal vector data. Noting that the timestamp graph may suffer from efficiency degradation when processing queries with long time intervals, we introduce a neighbor list precomputation strategy for large subtrees in the historic neighbor tree. By leveraging these precomputed neighbors, the query time in the compressed timestamp graph becomes independent of the query interval length, while maintaining the same space complexity as the original design. Table I summarizes the complexity of algorithms specifically tailored

to support vector searches under temporal validity constraints. Our major contributions are summarized as follows:

- We investigate the approximate nearest neighbor search over temporal vector data, a new type of queries in vector databases for emerging AI applications.
- We propose the timestamp graph, a novel structure to manage valid vectors across timestamps by a single index. It enables accurate and efficient TANNS queries with fast index updating and low index storage overhead.
- We design the historic neighbor tree, which compresses the timestamp graph to the limit, achieving the same space complexity as index for a single timestamp.
- We extend the timestamp graph to support IANNS queries and design the strategy of neighbor list precomputation to boost efficiency for IANNS, achieving search time complexity independent the query interval length.
- Extensive experiments show that our solution yields a recall rate of over 99% on five datasets, while improving the query efficiency for TANNS and IANNS by $3.3\times$ to $138.1\times$ over the state-of-the-arts.

II. PROBLEM STATEMENT

We first review the nearest neighbor search, a primitive in vector databases [2], before defining our problem of approximate nearest neighbor search over temporal vector data.

Definition 1 ((Exact) k Nearest Neighbor (kNN)): Given N high-dimensional vectors $\mathcal{D} = \{u_1, u_2, \dots, u_N\}$ and a query vector q of the same dimension, the (exact) kNN query $kNN(\mathcal{D}, q, k)$ retrieves a subset of k vectors that are closest to q , i.e., $\forall u \in kNN(\mathcal{D}, q, k)$ and $\forall v \in \mathcal{D} \setminus kNN(\mathcal{D}, q, k)$, $dis(q, u) \leq dis(q, v)$, where $dis(\cdot, \cdot)$ is the distance function.

Unlike its low-dimensional counterpart [21], kNN on high-dimensional data often prioritizes *efficiency* over *accuracy* to align with the targeting vector database applications [1], [2], which results in the following query optimization scope:

- *Approximate Queries.* Most studies [22] focus on approximate nearest neighbor search (ANNS) with its accuracy assessed by the *recall rate*, a key metric for downstream vector database tasks such as text retrieval [7] and recommendation [23].
- *Dedicated Indexes.* Data in vector database applications is not only high-dimensional but also large-scale, making indexing indispensable for efficient queries. Graph-based indexes, e.g., the hierarchical navigable small-world graph (HNSW) prevail in ANNS queries on high-dimensional vectors [19], [20]. They yield high accuracy by visiting merely $O(\log N)$ vectors.

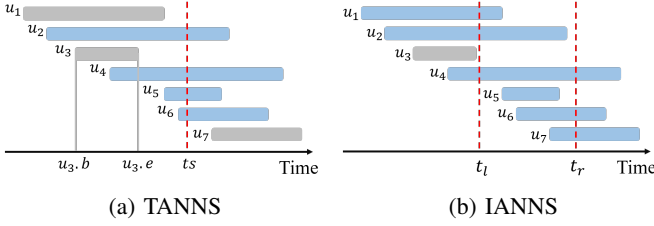


Fig. 1: Example of TANNS and IANNS.

We follow the above scope yet extend it with *time attributes* in both the vectors and the query. In the following, we define two problems: Timestamp Approximate Nearest Neighbor Search (TANNS), which retrieves vectors valid at a specific timestamp, and Interval Approximate Nearest Neighbor Search (IANNS), which retrieves vectors whose validity intervals intersect with a query time interval.

Definition 2 (Timestamp Approximate Nearest Neighbor Search (TANNS)): Let $\mathcal{D} = \{u_1, u_2, \dots, u_N\}$ be N high-dimensional vectors. Each vector u_i is associated with two timestamps, $u_i.s$ and $u_i.e$ ($u_i.s < u_i.e$), representing the start and end of its validity. Let q be a query vector of the same dimension, which is associated with a timestamp ts . The timestamp approximate nearest neighbor search $TANNS(\mathcal{D}, ts, q, k)$ returns a subset of k valid vectors that are approximately closest to q .

Definition 3 (Interval Approximate Nearest Neighbor Search (IANNS)): Let $\mathcal{D} = \{u_1, u_2, \dots, u_N\}$ be N high-dimensional vectors with start and end timestamps. The query object q is associated with a time interval $ti = [t_l, t_r]$ ($t_l \leq t_r$). The interval approximate nearest neighbor search $IANNS(\mathcal{D}, ti, q, k)$ returns a subset of k valid vectors that are approximately closest to q , where a vector is valid if its time range intersects with the query interval.

For example, as illustrated in Fig. 1, each vector is associated with a validity time range defined by its start and end timestamps. $\{u_2, u_4, u_5, u_6\}$ are valid in TANNS as their validity time range cover the query timestamp ts , while all vectors except u_3 are valid in IANNS, since their validity time range intersect with the query interval $[t_l, t_r]$.

These new queries captures the data dynamics and time-aware queries in emerging vector database applications, e.g., memory controls and time references in chat bots [8]. Without loss of generality, we assume discrete timestamps. At each timestamp, exactly one vector becomes valid or invalid, and we consider a horizon from 1 to T , where T refers to the current timestamp. We evaluate query accuracy by recall rate $\frac{|r \cap r^*|}{k}$, where r is the result of the TANNS and IANNS query, and r^* is the result of the exact kNN query over all the valid vectors at the query timestamp or the query time interval. We aim to optimize the *efficiency* of TANNS and IANNS queries while retaining high accuracy over *large-scale, dynamic* data. Hence, we explore designs that support efficient index *update* and *search* in terms of both *time* and *memory*.

The rest of paper is organized as follows. We first focus on the TANNS queries, introducing naive graph-based method (Sec. III), timestamp graph (Sec. IV) and compressed times-

Algorithm 1: HNSW Search

Input: HNSW index G , query q , query number k

Output: a set of k ANN of q

```

1  $ep \leftarrow$  entry point of  $G$ ;
2 Initialize  $pool$  and  $ann$  as  $\{ep\}$ ;
3 while  $pool$  is not empty do
4    $u \leftarrow$  pop closest point to  $q$  in  $pool$ ;
5    $v \leftarrow$  point furthest from  $q$  in  $ann$ ;
6   if  $dis(q, v) > dis(q, u)$  then break;
7   foreach unvisited point  $o \in G[u]$  do
8     Mark  $o$  as visited;
9      $v \leftarrow$  point furthest from  $q$  in  $ann$ ;
10    if  $|ann| < k'$  or  $dis(q, o) < dis(q, v)$  then
11      Insert  $v$  into  $ann$  and  $pool$ ;
12      if  $|ann| > k'$  then
13        Remove point furthest from  $q$  in  $ann$ ;
14 return  $k$  points closest to  $q$  in  $ann$ ;
```

tamp graph (Sec. V) for the problem. We then extend our solution to IANNS queries in Sec. VI. Finally, we conduct the experimental evaluation in Sec. VII, review existing studies in Sec. VIII and conclude in Sec. IX.

III. NAIVE GRAPH-BASED TANNS

This section presents a naive solution to TANNS by adopting graph-based indexes. We first review HNSW, a popular graph-based index for ANNS, and then apply it to TANNS.

A. Graph-based ANNS and HNSW

Graph-based indexes are widely adopted for efficient ANNS [20]. Given a set of vectors $\mathcal{D}$, a graph-based index constructs a proximity graph G for $\mathcal{D}$, where each point in G corresponds to a vector in $\mathcal{D}$, and each edge connects a pair of points satisfying certain neighborhood conditions. Then the ANNS can be performed by greedy routing towards the query vector following the graph. These indexes achieve high search efficiency and accuracy since their structures effectively capture the nearby relation between high-dimensional vectors [22].

The hierarchical navigable small-world graph (HNSW) is a popular graph-based index in the academia and industry [19]. It employs a multilayer graph structure and performs searches from the top layer down to the base layer. We briefly review HNSW's *search* and *construction* algorithms below. For simplicity, we only explain the operations at the base layer. Operations at other layers are similar.

- **HNSW Search.** Searching over the HNSW index follows the greedy routing principle, yet expands the search scope from k to k' (where $k' > k$ is a parameter) to avoid local minima and boost query accuracy (see Algorithm 1). The search begins from a predefined entry point ep and maintains two lists: $pool$ for candidate points and ann for potential nearest neighbors. In each iteration, it picks point v from $pool$ that is closest to q and adds v 's neighbors to $pool$. The search terminates when ann reaches the size of k' (*efSearch* in [19]) and all points in ann are closer to q than those in $pool$. Finally, k points in ann that are closest to q are returned as query result.

Algorithm 2: HNSW Construction

Input: dataset $\mathcal{D}$

```

1 Initialize an empty graph  $G$ ;
2 foreach point  $o \in \mathcal{D}$  do
3    $cand \leftarrow \text{Search}(G, o, M')$ ;
4    $G[o] \leftarrow \text{Select-Nbrs}(o, cand, M)$ ;
5   foreach point  $u \in G[o]$  do
6     Insert  $o$  to  $G[u]$ ;
7     if  $u$  has more than  $M$  neighbors then
8        $G[u] \leftarrow \text{Select-Nbrs}(u, G[u], M)$ ;
9 Function  $\text{Select-Nbrs}(o, cand, M)$ :
10  Sort  $cand$  by ascending distance to  $o$ ;
11   $nbr \leftarrow \emptyset$ ;
12  foreach point  $u \in cand$  do
13    if  $u$  is not dominated by points in  $nbr$  then
14      Insert  $u$  into  $nbr$ ;
15    if  $|nbr| = M$  then break;
16  return  $nbr$ ;

```

- **HNSW Construction.** The HNSW index is built via incremental insertion (see Algorithm 2). To insert a point o , we retrieve M' (*efConstruction* in [19]) candidate neighbors and select M of them as its final neighbors $G[o]$ (lines 3-4). The same neighbor selection function (i.e., *Select-Nbrs*) is applied to points in $G[o]$ to ensure that the number of neighbors is within M (lines 5-8). The neighbor selection strategy used in HNSW is heuristic, which prioritizes points closer to o and excluding dominated points (lines 9-16). A point u is dominated by v if $dis(o, v) < dis(o, u)$ and $dis(u, v) < dis(o, u)$.

The time complexity of searching N points by the HNSW index is $O(M \log N)$ since it visit $O(\log N)$ points [19] and access their M neighbors. The time complexity for HNSW index construction is $O(MN \log N)$, as each point is inserted sequentially, and each insertion involves a search operation which takes $O(M \log N)$ time. The space complexity for the HNSW index is $O(MN)$, as M neighbors are stored per point.

B. Adopting HNSW Index for TANNS Query

Basic Idea. Although HNSW is efficient for ANNS, it assumes a static vector set $\mathcal{D}$, whereas the *valid* vector set varies over time in TANNS. Hence, an intuitive solution to TANNS is to build one HNSW index for the valid vectors at each timestamp, resulting in T distinct HNSW indexes $G_1, G_2, \dots, G_T$. Each G_t is constructed with all valid vectors at timestamp t via Algorithm 2. For a given query $TANNS(\mathcal{D}, t, q, k)$, we pick the corresponding index G_t , on which we search for k approximate nearest vectors via Algorithm 1.

Complexity Analysis. Updating the index requires rebuilding an HNSW graph from scratch whenever the valid-point set changes, resulting in an update complexity of $O(MN \log N)$. Besides, maintaining HNSW indexes for all timestamps incurs a total space cost of $O(MN^2)$. The naive method is inefficient in both update and storage overhead, especially for practical applications containing millions of vectors.

IV. TIMESTAMP GRAPH FOR TANNS

This section introduces TANNS queries based on the timestamp graph, a novel structure that supports fast index updating and consumes low index storage. We first present the structure of the timestamp graph, then explain its search and construction algorithms, and finally analyze its complexity.

A. Timestamp Graph

Rationale. Data dynamics in practical vector database applications are mild [24], [25], which implies that the valid vector sets at nearby timestamps largely overlap. Hence, it is unnecessary to maintain a separate index per timestamp, a key efficiency bottleneck in the naive solution. We take advantage of such *temporal locality* and manage valid vectors across all T timestamps by a single structure, as explained below.

Structure. A timestamp graph $\mathcal{TG}$ is a proximity graph defined as in Sec. III-A yet over all historic valid vector sets $\mathcal{D}(1), \dots, \mathcal{D}(T)$. It can be considered as aggregating the T per-timestamp HNSW indexes into a single graph by removing the unchanged valid vectors between adjacent timestamps. Unlike the HNSW index that only stores the current neighbor list for each point, the timestamp graph tracks neighbor lists for all historic timestamps. By scanning the historic neighbor lists, it can trace the neighbors of a point at any timestamp.

B. TANNS Query on Timestamp Graph

To process a query $TANNS(\mathcal{D}, t, q, k)$ with a timestamp graph $\mathcal{TG}$, we search for the approximate nearest neighbors in a similar way to Algorithm 1. The main difference is that when visiting point u in line 7, its neighbors $G[u]$ are not fixed but dependent on the query timestamp t . The neighbors of u at timestamp t can be obtained by a binary search on u 's historic neighbor list. All other steps remains the same as standard HNSW search. The algorithm yields high accuracy and efficiency since it specifically searches within the graph built using vectors valid at the query timestamp.

C. Timestamp Graph Construction

In the naive solution (Sec. III-B), an entire HNSW index for $\mathcal{D}(t)$ points is built from scratch using valid vectors per timestamp t . In contrast, the construction of the timestamp graph only involves one point per timestamp. However, it must handle not only point *insertion*, but also point *expiration*. We show that point expiration impairs the graph connectivity and thus the search accuracy, and solve the problem with *backup neighbors*, as explained below.

Need for Backup Neighbors. To update the timestamp graph at a specific timestamp t , if a new point becomes valid, we add it to the graph and connect it with M neighbors. If otherwise a point expires, we mark it as invalid and disconnect it from its current M neighbors. Then we append a new neighbor list (with the invalid point removed) to all the affected neighbors. Over time, the neighbor list of certain points may become empty, reducing the overall graph connectivity, which decreases the search accuracy [25]. We try to maintain the

Algorithm 3: Point Insertion

Input: timestamp graph $\mathcal{TG}$, point o , timestamp t

```

1  $cand \leftarrow \text{Search}(\mathcal{TG}, o, M')$ ;
2  $\mathcal{TG}[o], B[o] \leftarrow \text{Select-Nbrs}(o, cand, 2M)$ ;
3 Initialize  $o$ 's historic neighbor list with  $\mathcal{TG}[o]$ ;
4 foreach  $u \in \mathcal{TG}[o]$  do
5   if  $o$  is not dominated by points in  $\mathcal{TG}[u]$  and closer to  $u$ 
     than furthest points in  $\mathcal{TG}[u]$  then
6     Move the furthest point in  $\mathcal{TG}[u]$  into  $B[u]$ ;
7     Add  $o$  to  $\mathcal{TG}[u]$ ;
8     Append  $\mathcal{TG}[u]$  to the historic neighbor list of  $u$ ,
       associated with timestamp  $t$ ;
9   else
10    Add  $o$  to  $B[u]$ ;
11    if  $|B[u]| > M$  then
12      Remove the furthest point from  $B[u]$ ;
```

graph connectivity by introducing a backup neighbor list per point. Specifically, for each point u , we store M backup neighbors $B[u]$ in addition to M primary neighbors in $\mathcal{TG}[u]$. Consequently, when a point in $\mathcal{TG}[u]$ expires, certain point in $B[u]$ can take up its place, avoiding decreased connectivity. With the backup neighbors in mind, we now describe the algorithm details for handling point insertion and expiration to update the timestamp graph at a given timestamp t .

Point Insertion. Adding a point to the timestamp graph $\mathcal{TG}$ is similar to the insertion in the HNSW, except for an extra step to maintain the backup neighbor list (see Algorithm 3). Concretely, when a new point o is added, we search the current graph and retrieve M' candidate neighbors as in the HNSW, but select $2M$ points from the candidates. The M points closest to o are allocated to $\mathcal{TG}[o]$ as primary neighbors, while the remaining points are stored in $B[o]$ as backup neighbors. For any point u whose neighbor list $\mathcal{TG}[u]$ is modified after point insertion, we append a new version with timestamp t in u 's historic neighbor list (lines 4-12).

Point Expiration. When a point expires, we disconnect the expired point from its current neighbors and select new neighbors for these points (see Algorithm 4). Specifically, we process all neighbors u of o as follows: If o is a backup neighbor, we simply discard o from $B[u]$ (lines 2-3). If o is a primary neighbor, we replace o with a point from $B[u]$ (lines 7-9). In case the backup neighbor list $B[u]$ is empty, we reselect neighbors for u as if u were a newly added point (lines 11-12).

D. Complexity Analysis

For each point in timestamp graph, finding all neighbors at a given timestamp via binary search on the historic neighbor list takes $O(\log N)$ time, resulting in an overall search complexity of $O(\log^2 N)$. For graph updating, we perform a search operation at first, which takes $O(\log^2 N)$ time. Although a search may also be triggered when handling point expiration, this only occurs once every M updates in the worst case, since only one point expires each time. Thus the time complexity for index updating is $O(\log^2 N) + \frac{M \cdot O(\log^2 N)}{M} = O(\log^2 N)$.

Due to the locality of timestamp graph updating, both point insertion and expiration only affect M points, *i.e.*,

Algorithm 4: Point Expiration

Input: timestamp graph $\mathcal{TG}$, point o , timestamp t

```

1 foreach  $u \in \mathcal{TG}[o]$  do
2   if  $o \in B[u]$  then
3     Remove  $o$  from  $B[u]$ ;
4   else if  $o \in \mathcal{TG}[u]$  then
5     Remove  $o$  from  $\mathcal{TG}[u]$ ;
6     if  $B[u]$  is not empty then
7        $cand \leftarrow \mathcal{TG}[u] \cup B[u]$ ;
8        $\mathcal{TG}[u] \leftarrow \text{Select-Nbrs}(u, cand, M)$ ;
9       Remove all elements in  $\mathcal{TG}[u]$  from  $B[u]$ ;
10    else
11       $cand \leftarrow \text{Search}(\mathcal{TG}, u, M')$ ;
12       $\mathcal{TG}[u], B[u] \leftarrow \text{Select-Nbrs}(u, cand, 2M)$ ;
13    Append  $\mathcal{TG}[u]$  to the historic neighbor list of  $u$ ,
      associated with timestamp  $t$ ;
```

only the neighbor lists of these M points are modified, and we only store the new neighbor lists for them. Since each update requires $O(M^2)$ additional storage, the overall space complexity of the timestamp graph is $O(M^2 N)$.

V. COMPRESSING TIMESTAMP GRAPH

This section pushes the space complexity of the timestamp graph to the limit (*i.e.*, same as HNSW index for a single timestamp) by compressing and managing the neighbor lists in the timestamp graph via the historic neighbor tree. We explain its structure, introduce the algorithms for neighbor list reconstruction and compression, and analyze the complexity of the compressed timestamp graph.

A. Historic Neighbor Tree

Motivation. Although the timestamp graph reduces the index storage from $O(MN^2)$ to $O(M^2 N)$, there is still redundancy in the *neighbor lists* of the timestamp graph. As it is unlikely to replace the entire neighbor list at once, the same points tend to appear in multiple neighbor lists.

The design of historic neighbor tree is inspired by the classic interval tree [26], but it differs in two key aspects: (i) The historic neighbor tree manages points whose presence in the neighbor list are unknown in advance, unlike the interval tree, where both endpoints of intervals are predefined. (ii) The historic neighbor tree employs a bottom-up construction approach to ensure the tree remains balanced.

Structure. A historic neighbor tree HNT for a point $o \in \mathcal{D}$ is a binary tree with multiple nodes. Fig. 2 illustrates the structure of an internal node of a historic neighbor tree. It contains a unique node identifier, a timestamp t , and the points valid at t . The leaf nodes share a similar structure but lack the timestamp. Both node types maintain two sequences for the points they contain: one sorted by the points' start times and the other by their end times. Furthermore, a point valid in multiple nodes is stored only in the *highest node* where it is valid. In addition to the historic neighbor tree, we use L_{now} to record points in the current neighbor list. These points are sorted by their start

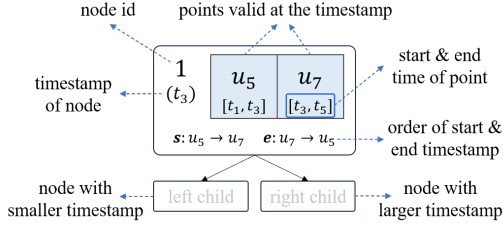
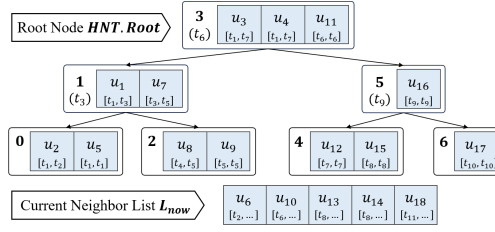


Fig. 2: Structure of a node in historic neighbor tree.

Fig. 3: Compressed historic neighbor list for point o .

Timestamp	$\mathcal{TG}[o]$
t_1	u_1, u_2, u_3, u_4, u_5
t_2	u_1, u_2, u_3, u_4, u_6
t_3	u_1, u_3, u_4, u_6, u_7
t_4	u_3, u_4, u_6, u_7, u_8
t_5	u_3, u_4, u_7, u_8, u_9
t_6	$u_3, u_4, u_6, u_{10}, u_{11}$
t_7	$u_3, u_4, u_6, u_{10}, u_{12}$
t_8	$u_6, u_{10}, u_{13}, u_{14}, u_{15}$
t_9	$u_6, u_{10}, u_{13}, u_{14}, u_{16}$
t_{10}	$u_6, u_{10}, u_{13}, u_{14}, u_{17}$
t_{11}	$u_6, u_{10}, u_{13}, u_{14}, u_{18}$

TABLE II: Historic neighbor lists for point o .**Algorithm 5: Reconstruct Neighbor List (Timestamp)**

Input: historic neighbor tree HNT , timestamp t
Output: neighbor list at timestamp t

- 1 Initialize L_t with points in L_{now} and valid at t ;
- 2 Node $n \leftarrow HNT.Root$;
- 3 **while** n is not leaf **do**
- 4 Add points in node n and valid at t to L_t ;
- 5 **if** $n.t = t$ **then return** L_t ;
- 6 **if** $n.t < t$ **then** $n \leftarrow n.left$;
- 7 **else** $n \leftarrow n.right$;
- 8 For node n , add all valid points at t to L_t ;
- 9 **return** L_t ;

times. Once points are removed from L_{now} , they are added to the historic neighbor tree.

Next, we explain how to (i) reconstruct the neighbor list at a given timestamp from the historic neighbor tree, and (ii) construct the historic neighbor tree from all neighbor lists.

B. Neighbor List Reconstruction

We outline in Algorithm 5 the procedure to reconstruct the neighbor list L_t at a given timestamp t from the historic neighbor tree. We first scan the current neighbors L_{now} and add points valid at t until encountering a point with a start time later than t (lines 2-5). Next, we start from the root and traverse the historic neighbor tree (lines 6-12). At each node, valid points are added to L_t . If t matches the node's timestamp, traversal stops as no valid points exist in lower nodes. Otherwise, traversal continues to the left or right child, depending on the relation between t and the node's timestamp. Reconstruction is completed after reaching a leaf node and including all valid points stored in it.

Example 2: Suppose we reconstruct L_{t_2} , the neighbor list at timestamp t_2 . We first scan the current neighbor list and add u_6 , as it is valid at t_2 . Then we stop at u_{14} since its start time exceeds t_2 . Afterwards, we traverse the historic neighbor tree from the root node, adding u_3 and u_4 as they are valid at t_2 . We proceed to node 1 (since $t_2 < t_6$), adding u_1 , then to node 0, where u_2 is the only valid point. In the end, we reconstruct L_{t_2} as $\{u_1, u_2, u_3, u_4, u_6\}$.

C. Historic Neighbor Tree Construction

We now describe how to compress the neighbor lists of a point o into a historic neighbor tree. We dynamically build the historic neighbor tree since the neighbors at each timestamp

Algorithm 6: Append Neighbor List

Input: historic neighbor tree HNT , the latest neighbor list L_t

- 1 **foreach** point $p \in L_{now} \setminus L_t$ **do**
- 2 **if** $p \notin HNT$ **then** // Adding New Point
- 3 Node $n_c \leftarrow$ the newest leaf node of HNT ;
- 4 **foreach** n in path from $HNT.Root$ to n_c **do**
- 5 **if** $n = n_c$ or p is valid at $n.t$ **then**
- 6 Insert p into n and **break**;
- 7 **if** n_c has more than μ points **then**
- 8 Subtree $st_c \leftarrow$ the maximal complete binary tree containing n_c ;
- 9 Create internal node n_i , with st_c as its left subtree and t as its timestamp;
- 10 Create leaf node n_l as right sibling of n_c ;
- 11 **if** n_i is higher than $HNT.Root$ **then**
- 12 $HNT.Root \leftarrow n_i$;
- 13 **else** // Adjusting Existing Point
- 14 Find node n_p which p located in;
- 15 **foreach** n in path from $HNT.Root$ to n_p **do**
- 16 **if** p is valid at $n.t$ **then**
- 17 Move p from n_p into n and **break**;
- 18 $L_{now} \leftarrow L_t$;

are unknown in advance. Upon receiving the latest neighbor list at t , we append it to the existing historic neighbor tree.

Algorithm 6 outlines the procedures. Consider the points in $L_{now} \setminus L_t$, which have just been removed from the current neighbor list. We first check whether point p already exists in the historic neighbor tree. If not, we execute the routine for adding a new point to the historic neighbor tree (lines 3-12). Otherwise, we simply adjust the position of the existing point in the tree (lines 14-17). Finally, we assign the current neighbor list to L_{now} (line 18).

D. Complexity Analysis

Let N denote the number of vectors and M denote the number of neighbors per point. The time complexity for searching and updating the compressed timestamp graph remains $O(\log^2 N)$, identical to that of the original timestamp graph. In addition, the space complexity of the compressed timestamp graph is $O(MN)$, which has reached the optimal, as the space complexity of HNSW index at a single timestamp is also $O(MN)$. A detailed complexity analysis is provided in the Sec. X-A1.

VI. EXTENDING TO IANNS

In this section, we extend our scope to the problem of Interval Approximate Nearest Neighbor Search (IANNS), where the query vectors are associated with a time interval instead of a single timestamp, and aim to retrieve the approximate nearest neighbors valid at the given interval. We first present the graph-based method for IANNS queries, then show the method of supporting IANNS queries with timestamp graph and compressed timestamp graph.

A. Naive Graph-based IANNS

A naive solution to IANNS is to build an HNSW graph for every possible time interval. Specifically, for each pair of timestamps $t_1 \in [1, T], t_2 \in [t_1, T]$, we construct an HNSW index $G_{[t_1, t_2]}$ using all vectors in $\mathcal{D}([t_1, t_2])$, i.e., vectors whose valid time range intersect with $[t_1, t_2]$. This results in $\frac{1}{2}T(T+1)$ distinct HNSW indexes in total. For a query $IANNS(\mathcal{D}, ti, q, k)$, where $ti = [t_l, t_r]$, we resort to the corresponding index $G_{[t_l, t_r]}$ and perform a standard HNSW search (i.e., Algorithm 1) to retrieve the top k closest vectors.

Complexity Analysis. Although the method maintains $O(M \log N)$ searching time, it builds $O(N^2)$ HNSW indexes and incurs $O(MN^3)$ space complexity, which is unacceptable as practical applications may deal with datasets with millions of vectors. Thus, in the following, we illustrate the method of supporting IANNS queries efficiently with timestamp graph, which has much lower space overhead.

B. Timestamp Graph for IANNS

The structure of the timestamp graph can be naturally extended to support IANNS queries. For TANNS queries, we perform greedy routing in the timestamp graph while only considering edges that connect points valid at the query timestamp. Similarly, to process an IANNS query with an associated time interval $[t_l, t_r]$, we perform greedy routing in the timestamp graph, but restrict traversal to edges that connect points valid during the query interval $[t_l, t_r]$.

The search procedure resembles Algorithm 1, with the key difference that when visiting a point u (line 7), its neighbors $G[u]$ are not fixed, but instead depend on the query interval. In the timestamp graph, the neighbors of point o for any timestamp are stored in its historic neighbor list $\mathcal{TG}[o]$. To determine the neighbors valid during $[t_l, t_r]$, we retrieve the relevant entries from $\mathcal{TG}[o]$ that overlap with the query interval, and then apply the *Select-Nbrs* function to select M candidate neighbors for the current iteration.

Complexity Analysis. The algorithm requires $O(MI + \log n)$ time to collect candidate neighbors over I timestamps, where I denotes the length of the query interval, followed by $O(M^2I)$ time to select M neighbors from these candidates. Since the search visits $O(\log N)$ points, the overall IANNS query complexity on the timestamp graph is $O(M^2I \log N + \log^2 N)$, which grows with the interval length I . Since the structure and construction procedure of the timestamp graph remain unchanged, its update and space complexities also remain $O(\log^2 N)$ and $O(M^2N)$, respectively.

Algorithm 7: Reconstruct Neighbor List (Interval)

Input: historic neighbor tree HNT , time interval $[t_l, t_r]$
Output: neighbor list during time interval $[t_l, t_r]$

- 1 Initialize $L_{[t_l, t_r]}$ with points in L_{now} valid at $[t_l, t_r]$;
- 2 $buf \leftarrow [HNT.Root]$;
- 3 **while** buf is not empty **do**
- 4 $n \leftarrow$ pop the first node in buf ;
- 5 $st_n \leftarrow$ the subtree rooted at node n ;
- 6 **if** $st_n.size < M$ **then**
- 7 Add points in n and valid at $[t_l, t_r]$ to $L_{[t_l, t_r]}$;
- 8 **else**
- 9 Add points in $st_n.pnl$ and valid at $[t_l, t_r]$ to $L_{[t_l, t_r]}$;
- 10 **if** n is leaf or $[st_n.t_{min}, st_n.t_{max}] \subseteq [t_l, t_r]$ **then**
- 11 **continue**;
- 12 **if** $t_l \leq n.t$ **then** Add $n.left$ to buf ;
- 13 **if** $t_r \geq n.t$ **then** Add $n.right$ to buf ;
- 14 $L_{[t_l, t_r]} \leftarrow \text{Select-Nbrs}(L_{[t_l, t_r]}, o, M)$;
- 15 **return** $L_{[t_l, t_r]}$;

Importantly, the search complexity depends directly on I , and in the worst case (large intervals) can degrade to $O(M^2N \log N)$. This motivates the use of the compressed timestamp graph, which reduces space overhead and yields query complexity that is *independent* of the interval length I .

C. Compressed Timestamp Graph for IANNS

As discussed in Sec. VI-B, the efficiency of IANNS queries critically depends on the computation of neighbor lists over the query interval. This challenge becomes more pronounced for long intervals, where the number of candidate neighbors can grow substantially, leading to high reconstruction cost and degraded query efficiency. To address this challenge, we further optimize the compressed timestamp graph by introducing a neighbor list precomputation strategy within the historic neighbor tree. The key idea is to leverage the hierarchical organization of the tree to summarize representative neighbors for large subtrees in advance, thereby avoiding scanning a large number of candidates during query processing. In this section, we begin by introducing neighbor list precomputation method, followed by an IANNS algorithm that achieves time complexity independent of the interval length.

Neighbor List Precomputation in Historic Neighbor Tree.

To support efficient interval neighbor list computation, we augment each node in the historic neighbor tree with subtree-level metadata. Specifically, each node n maintains the following information for the subtree rooted at n : (i) *size*, the number of points in the subtree; (ii) t_{min} and t_{max} , the earliest starting timestamp and the latest ending timestamp among all points in the subtree; and (iii) *pnl*, the pre-computed neighbor list selected from all points in the subtree. The precomputation proceeds incrementally during updates. When a point expires, we traverse its historic neighbor tree to identify subtrees whose sizes exceed M , and process them in ascending order of their root heights. For each such subtree, we collect all contained points and apply *Select-Nbrs* to generate a neighbor list of size M , which is stored as its *pnl*. To avoid redundant computation

for higher-level subtrees, we reuse the *pnl* lists of their child subtrees instead of rescanning all points.

For example, in node 1 of Fig. 4, we record $size = 6$, $t_{min} = t_1$ (from u_1, u_2, u_5), and $t_{max} = t_5$ (from u_7, u_8, u_9). Since the subtree size exceeds the threshold $M = 5$, we precompute its neighbor list by applying *Select-Nbrs* to all points in the subtree, yielding $pnl = \{u_1, u_2, u_7, u_8, u_9\}$. This precomputed list can be reused to accelerate neighbor list reconstruction during subsequent IANNS queries.

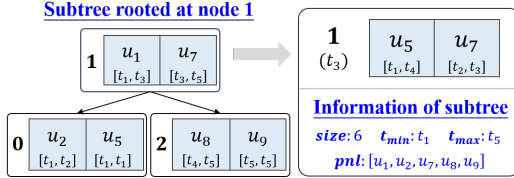


Fig. 4: Neighbor list precomputation in historic neighbor tree.

IANNS Query on Compressed Timestamp Graph. The IANNS query over the compressed timestamp graph follows the same procedure as Algorithm 1, except that the neighbors in line 7 are restricted to those valid within the query interval $[t_l, t_r]$. Leveraging the precomputed neighbor lists in the historic neighbor tree, we can efficiently reconstruct the neighbor list for a query interval, as described in Algorithm 7.

Similar to reconstruction at a single timestamp, we first scan the current neighbor list L_{now} and add points whose validity intervals intersect $[t_l, t_r]$ (line 1). We then traverse the historic neighbor tree starting from the root, maintaining a FIFO buffer queue *buf* of nodes to visit. In each iteration, we pop a node n from *buf* and process it. If the subtree rooted at n has size smaller than M , we scan the points stored in node n and add those valid in $[t_l, t_r]$ to $L_{[t_l, t_r]}$. Otherwise, we directly use its precomputed neighbor list *pnl* to retrieve candidates valid in $[t_l, t_r]$ (lines 6-9). Moreover, if the timestamp range of this subtree, i.e., $[st_n.t_{min}, st_n.t_{max}]$, is fully contained in $[t_l, t_r]$, we skip exploring its child nodes; otherwise, we continue the traversal by adding its children to *buf*. Specifically, we append the left child of node n to *buf* if n 's timestamp is no smaller than t_l , and append the right child if it is no larger than t_r (lines 10-13). The process terminates when *buf* is empty, after which we select M neighbors from $L_{[t_l, t_r]}$ using *Select-Nbrs* as the final neighbor list for the interval (lines 14-15).

Example 3: Suppose we aim to reconstruct $L_{[t_1, t_7]}$, the neighbor list for the time interval $[t_1, t_7]$. We first scan the current neighbor list and add u_6 and u_{10} to $L_{[t_1, t_7]}$, since their valid periods intersect with $[t_1, t_7]$. We then visit the root node 3 and adding u_3, u_4, u_{11} to $L_{[t_1, t_7]}$. Since the timestamp of the root node t_6 lies between t_1 and t_7 , we need to visit both the left child and the right child of node 3. For node 1, since its time range $[t_1, t_5]$ is fully contained in $[t_1, t_7]$, we add all the valid in the *pnl* to $L_{[t_1, t_7]}$. For node 5, all points within it are invalid during $[t_1, t_7]$, so we proceed to its left child, node 4, which contains the valid point u_{12} . In the end, we obtain $L_{[t_1, t_7]} = \{u_3, u_4, u_6, u_7, u_8, u_9, u_{10}, u_{11}, u_{12}\}$ and use *Select-Nbrs* to select 5 points from the list as the neighbors for the time interval. $\square$

Complexity Analysis. In contrast to the timestamp graph, performing IANNS on the compressed timestamp graph takes $O(M \log^2 N)$ time, independent of the query interval length I . Moreover, precomputing neighbor lists for large subtrees does not affect the update or space complexity of the compressed timestamp graph, which remain $O(\log^2 N)$ and $O(MN)$, respectively. A detailed complexity analysis is provided in Sec. X-A2.

VII. EXPERIMENTAL STUDY

A. Experiment Setup

Datasets with Synthetic Timestamps. We use three widely adopted high-dimensional vector datasets.

- **SIFT [27]:** Each point represents an 128-dimensional image descriptor, extracted from INRIA Holidays images using the Scale-Invariant Feature Transform algorithm.
- **GIST [27]:** Each point is a 960-dimensional image descriptor derived from the Holidays image set and Flickr images, capturing global image properties.
- **GloVe [28]:** Each data point is a 200-dimensional word vector generated from tweets using the Global Vectors for Word Representation model.

Each dataset contains 1 million data vectors. We use Euclidean distance to measure similarity in SIFT and GIST datasets, and cosine distance for GloVe dataset.

To simulate diverse temporal patterns, we assign data vectors to four types of following synthetic temporal patterns:

- **SV:** Each Vector has Short valid time range ($\leq 0.05T$).
- **LV:** Each Vectors with Long valid time ranges ($\geq 0.5T$).
- **MV:** Vectors with Mixed valid time ranges (half $\leq 0.05T$ and half $\geq 0.5T$).

Datasets with Real Timestamps. We also evaluate our methods on two datasets with real-world temporal information.

- **TWiki [4]:** This dataset contains Wikipedia items from September 2024 to February 2025. We use TWiki-Diffsets [4] to extract start and end timestamps for each text chunk and embed them with DeBERTa [29], yielding 1.5M vectors of 1024 dimensions.
- **MIND [30]:** This dataset contains news articles from October to November 2019. Following [31], each article's validity interval is defined by its first click and 95%-click timestamps. We embed the articles using Longformer [32], yielding 231K vectors of 768 dimensions.

Query Workloads. We use standard query vectors from each dataset, each query is associated with one of the following temporal conditions:

- **TQ:** Each Query is attached with a single Timestamp selected uniformly from $[1, T]$ (i.e., TANNs).
- **SQ:** Each Query is attached with a randomly selected Short time interval (i.e., IANNS with $I = 0.005T$).
- **MQ:** Each Query is attached with a randomly selected Medium time interval (i.e., IANNS with $I = 0.05T$).
- **LQ:** Each Query is attached with a randomly selected Long time interval (i.e., IANNS with $I = 0.5T$).

The number of results required for each query is set to $k = 10$.

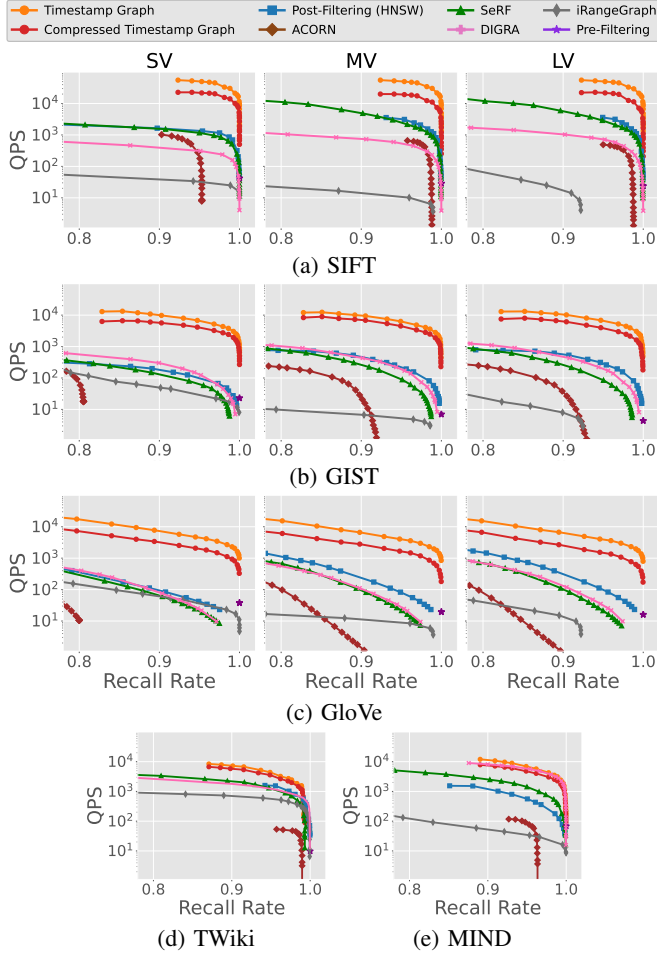


Fig. 5: Search performance of TANNS under different datasets and data patterns.

Baselines. We compare our method with following solutions.

- **Pre-Filtering** [24]: It first filters the dataset to retrieve all vectors valid at the query timestamp or interval, then scans the valid vectors to find vectors closest to the query.
- **Post-Filtering (HNSW)** [24]: It first searches the vector index (*i.e.*, HNSW), then filters the results based on validity at the query timestamp or interval.
- **ACORN** [15]: It supports ANNS with diverse query predicates. To adapt it for TANNS/IANNS, we assume validity periods are known in advance and set γ to the inverse of the average query selectivity.
- **SeRF** [14]: It is designed for range-filtering ANNS queries. We adapt it for our problem by querying the 1D segment graph with the start time as the attribute and apply post-filtering based on the end time.
- **DIGRA** [33]: It is designed for range-filter ANNS queries dynamic range-filtering ANNS. We adapt it in the same manner as SeRF.
- **iRangeGraph** [18]: It is designed for range-filtering ANNS queries, and we adapt its multiple-attributes version for our problem. For TANNS, the filtering condition is $u_{i.s} \leq t_s$ and $u_{i.e} \geq t_s$. For IANNS, the filtering condition is $u_{i.s} \leq t_r$ and $u_{i.e} \geq t_l$.

Implementation. All methods are implemented in C++ and compiled using GCC 9.4.0 with -Ofast optimization. For the

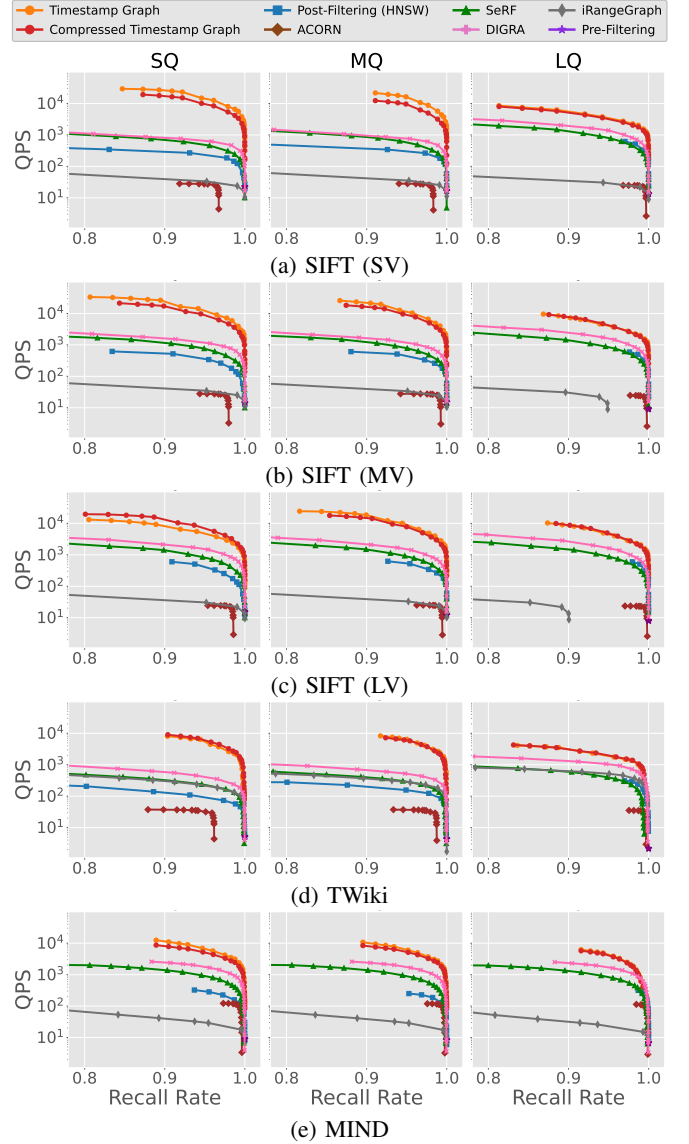


Fig. 6: Search performance of IANNS under different datasets and data patterns.

timestamp graph, we use the following default parameters: neighbor number $M = 16$, candidate neighbor number $M' = 200$ and leaf node size for historic neighbor tree $\mu = 8$. All the experiments are carried on a server with Intel Xeon(R) Gold 6240 2.60GHz CPU processors and 768GB RAM.

B. Experiments on Search Performance

We evaluate the search performance of our method on all five datasets. Following prior work, we use QPS–Recall plots to measure speed–accuracy trade-off [34].

1) *Search Performance of TANNS:* Fig. 5 presents the relationship between the QPS of TANNS and recall rate for all methods under three different datasets.

Comparison Between Methods. For all methods, QPS and recall rate are inversely related. Across all datasets and data patterns, both timestamp graph and compressed timestamp graph consistently outperform baseline algorithms in QPS at the same recall rate, achieving an advantage of $4.4\text{--}138.1\times$

over the existing solutions at the recall rate of 95%, with the highest improvement observed on the GloVe (SV) dataset.

Varying Data Patterns. Data patterns affect search performance by influencing the *selectivity* of the query timestamp, *i.e.*, the ratio of valid vectors at a given timestamp to the total vectors in $\mathcal{D}$. The performance of Pre-Filtering improve by $1.3\text{--}5.3\times$ in the short data pattern due to the reduced number of valid vectors at the query timestamp. In the GIST dataset and at the recall rate of 95%, the QPS of Post-Filtering and SeRF decreases from 260 and 84 to 74 and 57, respectively, when transitioning from the long to short data pattern. This decline occurs because lower selectivity forces these methods to expand their search scope to satisfy temporal constraints, thus reducing efficiency. In contrast, the influence of data pattern on our method is minimal. Fixing the recall rate at 95%, timestamp graph processes 3984 searches per second in GIST (LV) and 4149 searches per second in GIST (SV).

2) *Search Performance of IANNS*: Fig. 6 presents the result of performing IANNS queries on three different datasets under different data and query patterns.

Comparison Between Methods. Consistent with the observations from TANNS, all methods exhibit an inverse relationship between QPS and recall rate. Both timestamp graph and compressed timestamp graph maintain recall rates above 99% across all datasets and workloads. At a fixed recall rate of 95%, our methods consistently outperform the baseline algorithms in QPS, with the timestamp graph achieving an improvement of $3.4\text{--}27.6\times$ over the best baseline. For example, on the SIFT dataset with short data pattern and short query pattern, the timestamp graph attains QPS of 12,658, whereas the best baseline algorithm only reaches QPS of 459.

Varying Data and Query Patterns. Under the same query pattern, different data patterns primarily affect selectivity and therefore impact search performance. Baseline methods except for Pre-Filtering experience a noticeable drop in search performance under lower selectivity. Both of our methods consistently outperform all the baseline methods across varying data patterns. Across all data patterns on SIFT dataset, the timestamp graph delivers at least $3.3\times$ the QPS of the best baseline, while the compressed timestamp graph provides at least $3.4\times$ speed-up. This stability comes from selecting neighbors only from the valid points within the query interval. Similar to the data pattern, our methods maintain relatively stable performance across different query interval lengths and consistently outperform baseline methods. For example, on the TWiki and MIND datasets under different query patterns, the timestamp graph and compressed timestamp graph achieve up to $29.2\times$ and $28.6\times$ performance gains, respectively, over the best baseline.

C. Experiments on Index Construction

We evaluate the index construction performance of the timestamp graph using update throughput (the number of updates processed per second) and memory usage as metrics.

Comparison Between Methods. Fig. 7 presents the update throughput and memory usage of methods under various datasets and data patterns. The results show that the

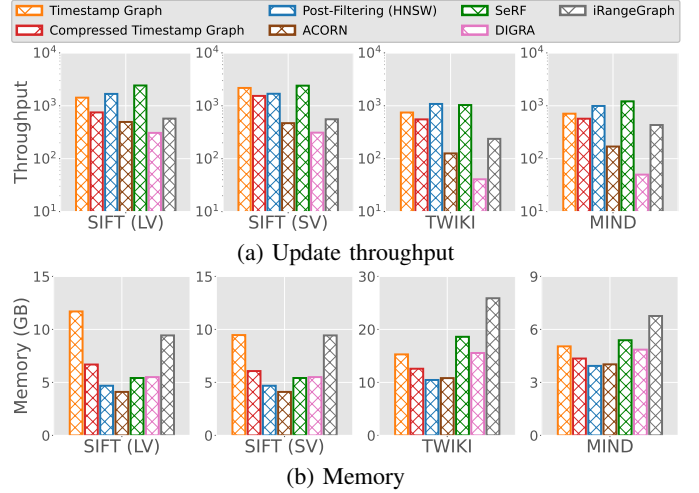


Fig. 7: Index construction under different datasets.

timestamp graph achieves update throughput comparable to that of the HNSW index (*i.e.*, post-filtering method), with a relative performance of $0.7\text{--}1.4\times$, while providing higher update throughput than most baselines across different settings. For memory usage, the compressed timestamp graph reduces storage overhead by up to 42.8% compared with the timestamp graph, achieving comparable or lower memory consumption than the baselines. On datasets with real timestamps, the compressed timestamp graph is more memory-efficient than SeRF, iRangeGraph, and DIGRA, achieving better performance while requiring only 48.6%–89.6% of their memory usage.

Varying Data Pattern. HNSW’s update throughput and memory usage remain unaffected by data patterns, as it does not account for time attributes during index construction. In contrast, the update throughput for the timestamp graph is influenced by the data pattern. This is because timestamp graphs for short patterns contain fewer valid vectors, which reduces the time required to search for candidate neighbors during index construction, thereby improving update throughput. For ACORN, SeRF, DIGRA, and iRangeGraph, data patterns have little impact on construction performance. This is because they either ignore timestamps during construction, or only use the start timestamp during construction and handle the end timestamp through post-filtering.

VIII. RELATED WORK

Approximate Nearest Neighbor Search. ANNS is a core operation in vector databases [1], [24], [35]. Indexes for ANNS mainly fall into three categories: partition-based [36]–[40], quantization-based [27], [41], and graph-based [19], [20], [42], [43], where the last one best balances search accuracy and efficiency [22], [34]. HNSW and NSG are two representative graph-based indexes but they struggle to support point deletion. FreshDiskANN [44] and Wolverine [45] support dynamic updates via graph reconnection. However, frequent point updates in TANNS/IANNS would incur substantial graph maintenance overhead, making them inefficient for our setting.

Another related line of research is hybrid approximate nearest neighbor search [46], which performs ANNS un-

der additional *numeric* or *categorical* filtering conditions. A branch closely related to our problem is range-filtered ANNS (RFANNS) [14], [18], [33], [47], [48], where each vector is associated with a single numeric attribute and queries specify a range over this attribute. In contrast, TANNS/IANNS associate each vector with two timestamps (begin and end timestamps) that define a validity interval, while the expiration timestamp may be unknown at insertion time, making the problem fundamentally different. Other hybrid ANNS methods focus on categorical filtering [16], [49], while some support both numeric and categorical filters [15]. Among these methods, we empirically compare against ACORN [15], SeRF [14], iRange-Graph [18] and DIGRA [33], which are representative state-of-the-art approaches adaptable to TANNS/IANNS queries.

Temporal Data Management. Temporal databases store and retrieve data records that changes over time [50]. There are many dedicated indexes for temporal data [51]. Among them, MVB-Tree [52], HV-Tree [53], Timeline Index [54], and LIT [55] are indexes designed for time-varied relational data. Other indexes are intended for querying spatial-temporal objects, such as HR-Tree [10], TPR-Tree [11] and MV3R-Tree [12]. Yet they struggle to support similarity searches in the high-dimensional space due to the “curse of dimensionality” [13].

IX. CONCLUSION

This paper defines the TANNS and IANNS problem and designs both time- and space-efficient solutions for these queries. We propose the timestamp graph to effectively manage valid vectors across all timestamps via a unified structure. We further develop the historic neighbor tree to compress neighbor lists in the timestamp graph and push its space complexity to the limit (same as index at a single timestamp). Besides, we extend timestamp graph to support queries which specified an interval and propose the optimization of neighbor list pre-computation to optimize the efficiency. Extensive evaluations show that our solution achieves up to $138.1\times$ higher QPS at 95% recall compared to prior proposals.

REFERENCES

- [1] J. Wang, X. Yi, R. Guo, H. Jin, P. Xu, S. Li, X. Wang, X. Guo, C. Li, X. Xu, K. Yu, Y. Yuan, Y. Zou, J. Long, Y. Cai, Z. Li, Z. Zhang, Y. Mo, J. Gu, R. Jiang, Y. Wei, and C. Xie, “Milvus: A purpose-built vector data management system,” in *International Conference on Management of Data*, 2021, pp. 2614–2627.
- [2] J. J. Pan, J. Wang, and G. Li, “Survey of vector database management systems,” *The VLDB Journal*, vol. 33, no. 5, pp. 1591–1615, 2024.
- [3] A. Asai, S. Min, Z. Zhong, and D. Chen, “Retrieval-based language models and applications,” in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*, 2023, pp. 41–46.
- [4] J. Jang, S. Ye, C. Lee, S. Yang, J. Shin, J. Han, G. Kim, and M. Seo, “Temporalwiki: A lifelong benchmark for training and evaluating ever-evolving language models,” in *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, 2022, pp. 6237–6250.
- [5] R. Campos, G. Dias, A. M. Jorge, and A. Jatowt, “Survey of temporal information retrieval and related applications,” *ACM Computing Surveys*, 2014.
- [6] J. Wallat, A. Jatowt, and A. Anand, “Temporal blind spots in large language models,” in *Proceedings of the 17th ACM International Conference on Web Search and Data Mining*, 2024, pp. 683–692.
- [7] V. Karpukhin, B. Oguz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W. Yih, “Dense passage retrieval for open-domain question answering,” in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, 2020, pp. 6769–6781.
- [8] B. Dhingra, J. R. Cole, J. M. Eisenschlos, D. Gillick, J. Eisenstein, and W. W. Cohen, “Time-aware language models as temporal knowledge bases,” *Trans. Assoc. Comput. Linguistics*, vol. 10, pp. 257–273, 2022.
- [9] G. Izacard, P. Lewis, M. Lomeli, L. Hosseini, F. Petroni, T. Schick, J. Dwivedi-Yu, A. Joulin, S. Riedel, and E. Grave, “Atlas: Few-shot learning with retrieval augmented language models,” *Journal of Machine Learning Research*, no. 251, pp. 1–43, 2023.
- [10] M. A. Nascimento and J. R. O. Silva, “Towards historical r-trees,” in *Proceedings of the 1998 ACM symposium on Applied Computing*, 1998, pp. 235–240.
- [11] S. Saltenis, C. S. Jensen, S. T. Leutenegger, and M. A. López, “Indexing the positions of continuously moving objects,” in *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data*, 2000, pp. 331–342.
- [12] Y. Tao and D. Papadias, “Mv3r-tree: A spatio-temporal access method for timestamp and interval queries,” in *Proceedings of 27th International Conference on Very Large Data Bases*, P. M. G. Apers, P. Atzeni, S. Ceri, S. Paraboschi, K. Ramamohanarao, and R. T. Snodgrass, Eds., 2001, pp. 431–440.
- [13] P. Indyk and R. Motwani, “Approximate nearest neighbors: towards removing the curse of dimensionality,” in *Proceedings of the thirtieth annual ACM symposium on Theory of computing*, 1998, pp. 604–613.
- [14] C. Zuo, M. Qiao, W. Zhou, F. Li, and D. Deng, “Serf: Segment graph for range-filtering approximate nearest neighbor search,” *Proc. ACM Manag. Data*, vol. 2, no. 1, pp. 69:1–69:26, 2024.
- [15] L. Patel, P. Kraft, C. Guestrin, and M. Zaharia, “ACORN: performant and predicate-agnostic search over vector embeddings and structured data,” *Proc. ACM Manag. Data*, vol. 2, no. 3, p. 120, 2024.
- [16] S. Gollapudi, N. Karia, V. Sivashankar, R. Krishnaswamy, N. Begwani, S. Raz, Y. Lin, Y. Zhang, N. Mahapatro, P. Srinivasan, A. Singh, and H. V. Simhadri, “Filtered-diskann: Graph algorithms for approximate nearest neighbor search with filters,” in *Proceedings of the ACM Web Conference 2023*, 2023, pp. 3406–3416.
- [17] J. Mohoney, A. Pacaci, S. R. Chowdhury, A. Mousavi, I. F. Ilyas, U. F. Minhas, J. Pound, and T. Rekatsinas, “High-throughput vector similarity search in knowledge graphs,” *Proc. ACM Manag. Data*, vol. 1, no. 2, pp. 197:1–197:25, 2023.
- [18] Y. Xu, J. Gao, Y. Gou, C. Long, and C. S. Jensen, “irangegraph: Improving range-dedicated graphs for range-filtering nearest neighbor search,” *Proc. ACM Manag. Data*, vol. 2, no. 6, pp. 239:1–239:26, 2024.
- [19] Y. A. Malkov and D. A. Yashunin, “Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 42, no. 4, pp. 824–836, 2020.
- [20] M. Wang, X. Xu, Q. Yue, and Y. Wang, “A comprehensive survey and experimental comparison of graph-based approximate nearest neighbor search,” *Proc. VLDB Endow.*, vol. 14, no. 11, pp. 1964–1978, 2021.
- [21] T. K. Sellis, N. Roussopoulos, and C. Faloutsos, “Multidimensional access methods: Trees have grown everywhere,” in *Proceedings of 23rd International Conference on Very Large Data Bases*, 1997, pp. 13–14.
- [22] W. Li, Y. Zhang, Y. Sun, W. Wang, M. Li, W. Zhang, and X. Lin, “Approximate nearest neighbor search on high dimensional data - experiments, analyses, and improvement,” *IEEE Trans. Knowl. Data Eng.*, vol. 32, no. 8, pp. 1475–1488, 2020.
- [23] A. Babenko and V. S. Lempitsky, “Efficient indexing of billion-scale datasets of deep descriptors,” in *2016 IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 2055–2063.
- [24] C. Wei, B. Wu, S. Wang, R. Lou, C. Zhan, F. Li, and Y. Cai, “Analyticdbv: A hybrid analytical engine towards query fusion for structured and unstructured data,” *Proc. VLDB Endow.*, vol. 13, no. 12, pp. 3152–3165, 2020.
- [25] Y. Xu, H. Liang, J. Li, S. Xu, Q. Chen, Q. Zhang, C. Li, Z. Yang, F. Yang, Y. Yang, P. Cheng, and M. Yang, “Spfresh: Incremental in-place update for billion-scale vector search,” in *Proceedings of the 29th Symposium on Operating Systems Principles*, 2023, pp. 545–561.
- [26] H. Edelsbrunner, “A new approach to rectangle intersections,” *International Journal of Computer Mathematics*, no. 3–4, pp. 209–219, 1983.
- [27] H. Jégou, M. Douze, and C. Schmid, “Product quantization for nearest neighbor search,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 33, no. 1, pp. 117–128, 2011.
- [28] J. Pennington, R. Socher, and C. D. Manning, “Glove: Global vectors for word representation,” in *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing*, 2014, pp. 1532–1543.
- [29] P. He, J. Gao, and W. Chen, “Debertav3: Improving deberta using electra-style pre-training with gradient-disentangled embedding shar-

ing,” in *The Eleventh International Conference on Learning Representations*, 2023.

- [30] F. Wu, Y. Qiao, J. Chen, C. Wu, T. Qi, J. Lian, D. Liu, X. Xie, J. Gao, W. Wu, and M. Zhou, “MIND: A large-scale dataset for news recommendation,” in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, 2020, pp. 3597–3606.
- [31] L. He, C. Han, A. Mukherjee, Z. Obradovic, and E. Dragut, “On the dynamics of user engagement in news comment media,” *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, 2020.
- [32] Huggingface, “Longformer.” [Online]. Available: https://huggingface.co/docs/transformers/model_doc/longformer
- [33] M. Jiang, Z. Yang, F. Zhang, G. Hou, J. Shi, W. Zhou, F. Li, and S. Wang, “DIGRA: A dynamic graph indexing for approximate nearest neighbor search with range filter,” *Proc. ACM Manag. Data*, vol. 3, no. 3, pp. 148:1–148:26, 2025.
- [34] M. Aumüller, E. Bernhardsson, and A. J. Faithfull, “Ann-benchmarks: A benchmarking tool for approximate nearest neighbor algorithms,” *Inf. Syst.*, vol. 87, 2020.
- [35] M. Douze, A. Guzhva, C. Deng, J. Johnson, G. Szilvasy, P.-E. Mazaré, M. Lomeli, L. Hosseini, and H. Jégou, “The faiss library,” *arXiv preprint arXiv:2401.08281*, 2024.
- [36] A. Gionis, P. Indyk, and R. Motwani, “Similarity search in high dimensions via hashing,” in *Proceedings of 25th International Conference on Very Large Data Bases*, 1999, pp. 518–529.
- [37] Q. Lv, W. Josephson, Z. Wang, M. Charikar, and K. Li, “Multi-probe LSH: efficient indexing for high-dimensional similarity search,” in *Proceedings of the 33rd International Conference on Very Large Data Bases*, 2007, pp. 950–961.
- [38] N. Sundaram, A. Turmukhametova, N. Satish, T. Mostak, P. Indyk, S. Madden, and P. Dubey, “Streaming similarity search over one billion tweets using parallel locality-sensitive hashing,” *Proc. VLDB Endow.*, vol. 6, no. 14, pp. 1930–1941, 2013.
- [39] Y. Tian, X. Zhao, and X. Zhou, “DB-LSH 2.0: Locality-sensitive hashing with query-based dynamic bucketing,” *IEEE Trans. Knowl. Data Eng.*, vol. 36, no. 3, pp. 1000–1015, 2024.
- [40] J. Wei, B. Peng, X. Lee, and T. Palpanas, “DET-LSH: A locality-sensitive hashing scheme with dynamic encoding tree for approximate nearest neighbor search,” *Proc. VLDB Endow.*, vol. 17, no. 9, pp. 2241–2254, 2024.
- [41] Y. Matsui, Y. Uchida, H. Jegou, and S. Satoh, “A survey of product quantization,” *ITE Transactions on Media Technology and Applications*, no. 1, pp. 2–10, 2018.
- [42] C. Fu, C. Xiang, C. Wang, and D. Cai, “Fast approximate nearest neighbor search with the navigating spreading-out graph,” *Proc. VLDB Endow.*, vol. 12, no. 5, pp. 461–474, 2019.
- [43] Y. Peng, B. Choi, T. N. Chan, J. Yang, and J. Xu, “Efficient approximate nearest neighbor search in multi-dimensional databases,” *Proc. ACM Manag. Data*, vol. 1, no. 1, pp. 54:1–54:27, 2023.
- [44] A. Singh, S. J. Subramanya, R. Krishnaswamy, and H. V. Simhadri, “Freshdiskann: A fast and accurate graph-based ANN index for streaming similarity search,” *CoRR*, vol. abs/2105.09613, 2021.
- [45] D. Liu, B. Zheng, Z. Yue, F. Ruan, X. Zhou, and C. S. Jensen, “Wolverine: Highly efficient monotonic search path repair for graph-based ANN index updates,” *Proc. VLDB Endow.*, vol. 18, no. 7, pp. 2268–2280, 2025.
- [46] M. Li, X. Yan, B. Lu, Y. Zhang, J. Cheng, and C. Ma, “Attribute filtering in approximate nearest neighbor search: An in-depth experimental study,” *Proc. ACM Manag. Data*, vol. 3, no. 6, pp. 1–26, 2025.
- [47] A. Liang, P. Zhang, B. Yao, Z. Chen, Y. Song, and G. Cheng, “UNIFY: unified index for range filtered approximate nearest neighbors search,” *Proc. VLDB Endow.*, vol. 18, no. 4, pp. 1118–1130, 2024.
- [48] Z. Peng, M. Qiao, W. Zhou, F. Li, and D. Deng, “Dynamic range-filtering approximate nearest neighbor search,” *Proc. VLDB Endow.*, vol. 18, no. 10, pp. 3256–3268, 2025.
- [49] Y. Cai, J. Shi, Y. Chen, and W. Zheng, “Navigating labels and vectors: A unified approach to filtered approximate nearest neighbor search,” *Proc. ACM Manag. Data*, vol. 2, no. 6, pp. 246:1–246:27, 2024.
- [50] C. S. Jensen and R. T. Snodgrass, “Temporal database,” in *Encyclopedia of Database Systems*, 2018.
- [51] B. Salzberg and V. J. Tsotras, “Comparison of access methods for time-evolving data,” *ACM Comput. Surv.*, vol. 31, no. 2, pp. 158–221, 1999.
- [52] B. Becker, S. Gschwind, T. Ohler, B. Seeger, and P. Widmayer, “An asymptotically optimal multiversion b-tree,” *VLDB J.*, vol. 5, no. 4, pp. 264–275, 1996.
- [53] R. Zhang and M. Stradling, “The hv-tree: a memory hierarchy aware version index,” *Proc. VLDB Endow.*, vol. 3, no. 1, pp. 397–408, 2010.

- [54] M. Kaufmann, A. A. Manjili, P. Vagenas, P. M. Fischer, D. Kossmann, F. Färber, and N. May, “Timeline index: a unified data structure for processing queries on temporal data in SAP HANA,” in *Proceedings of the ACM SIGMOD International Conference on Management of Data*, 2013, pp. 1173–1184.
- [55] G. Christodoulou, P. Boursos, and N. Mamoulis, “LIT: lightning-fast in-memory temporal indexing,” *Proc. ACM Manag. Data*, vol. 2, no. 1, pp. 20:1–20:27, 2024.



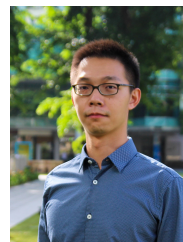
Yuxiang Wang is currently working toward the Ph.D degree in the School of Computer Science and Engineering, Beihang University. His major research interests include vector database and federated data management.



Ziyuan He is currently working toward the Ph.D degree in the School of Computer Science and Engineering, Beihang University. His major research interests include vector database.



Yongxin Tong received the Ph.D. degree in computer science and engineering from the Hong Kong University of Science and Technology in 2014. He is currently a professor in the School of Computer Science and Engineering, Beihang University. His research interests include federated learning, federated data management, spatio-temporal data analytics, privacy-preserving data analytics, crowdsourcing, uncertain data management, etc. He is a member of the IEEE.



Zimu Zhou received the B.E. degree from the Department of Electronic Engineering, Tsinghua University, Beijing, China, in 2011, and the Ph.D. degree from the Department of Computer Science and Engineering, Hong Kong University of Science and Technology, Hong Kong, in 2015. He is currently an assistant professor at the School of Data Science, City University of Hong Kong. His research focuses on mobile and ubiquitous computing. He is a member of the IEEE.



Zijie Ma is currently working toward the Master degree in the School of Computer Science and Engineering, Beihang University. His major research interests include vector database.



Yiman Zhong is currently working toward the Ph.D degree in the School of Computer Science and Engineering, Beihang University. Her major research interests include federated data management, privacy-preserving data analytics, etc.

X. APPENDICES

A. Complexity Analysis for Compressed Timestamp Graph

The section present the detailed complexity analysis for compressed timestamp graph.

1) *TANNS with Compressed Timestamp Graph*: We first characterize the structure of historic neighbor tree and then analyze the complexity of supporting TANNS queries with the compressed timestamp graph. We begin by showing the historic neighbor tree is balanced, whose height is logarithmic relative to the number of points it contains.

Lemma 1: If a historic neighbor tree has n points, then its height is at most $\lceil \log n \rceil$.

Proof: Suppose a historic neighbor tree has h levels, then its left subtree must be full (according to lines 7-10 in Algorithm 6), meaning that the subtree contains 2^{h-1} leaf nodes. Thus, the entire tree contains at least $\mu \cdot 2^{h-1}$ points, where the leaf node size μ is chosen to be greater than 2. Given that, a historic neighbor tree with 2^h points has a height of at most h , which completes our proof. ■

Time Complexity. We begin by analyzing the complexity of historic neighbor tree.

Lemma 2: The time complexity of neighbor list reconstruction of point u at a given timestamp is $O(\log n + M_r)$, where n is the number of points that ever appear in u 's history neighbor list and M_r is the size of the reconstructed neighbor list.

Proof: We prove the lemma by showing that almost all the points visited in Algorithm 5 appear in the reconstructed neighbor list. The exceptions are one point in the current neighbor list L_{now} and at most one points in each layer of the historic neighbor tree. The reasons are as follows:

- In lines 2-5, all points visited are valid until we encounter the first expired point, at which the scanning stops.
- In lines 7-14, we traverse the historic neighbor tree to add valid points from the nodes we visit. By using either the start time or end time order, we guarantee to access at most one expired point per node. If $t < n.t$, i.e., the query timestamp is smaller than the node's timestamp, we scan the list in the order of the start time and stop once we encounter an expired point. Similarly, if $t > n.t$, we scan the list in the order of the end time.

In summary, the time complexity of neighbor list reconstruction is the summation of the layer number in historic neighbor tree and the reconstructed neighbor list size, which is $O(\log n + M_r)$. ■

Our experimental results (see Sec. VII-B) indicate that the maximal size of the M_r is $1.31M$, which is slightly larger than the original size of neighbor list. Likewise, we can conclude that the time complexity for inserting into a point's historic neighbor tree is $O(\log n)$, since the insertion operation visits only one node at each layer.

We continue to analyze the compressed timestamp graph. Since the number of visited nodes during both search and update is $O(\log N)$, the time complexity for searching and updating the compressed timestamp graph is $O(\log^2 N)$.

Space Complexity. We proceed to analyze the space complexity for the compressed timestamp graph.

Lemma 3: The space complexity of compressed timestamp graph is $O(MN)$, where N is the size of dataset and M is the neighbor number for each point.

Proof: For a point with n points ever appearing in its historic neighbor list, the space complexity of its historic neighbor tree is $O(n)$, because each point in the historic neighbor list is stored in only one location, either in the current neighbor list L_{now} or in a node of the historic neighbor tree. Thus, the total space required for all points' neighbors is $\sum_{i=1}^N O(n_i)$, where n_i is the number of history neighbors of point u_i . This corresponds to the number of edges created during the index construction. Since each point creates at most $2M$ edges as it is inserted into the timestamp graph, the upper bound for $\sum_{i=1}^N n_i$ is $2MN$. Therefore, the space complexity of the compressed timestamp graph is $O(MN)$. ■

Note that the space complexity of the compressed timestamp graph has reached the optimal, as the space complexity of HNSW index at a single timestamp is $O(MN)$.

2) *IANNS with Compressed Timestamp Graph*: We now analyze the complexity of performing IANNS on compressed timestamp graph with the optimization of neighbor list pre-computation.

Time Complexity. In contrast to the timestamp graph, the time complexity of reconstructing neighbor list with compressed timestamp graph is independent of the length of the query interval, as shown in Lemma 4.

Lemma 4: The time complexity of reconstructing neighbor list for point u at given time interval with Algorithm 7 is $O(M \log n)$, where n is the number of points that appear in u 's history neighbor list.

Proof: As shown in Fig. 4, each node in the historic neighbor tree is associated with a time interval $[t_{min}, t_{max}]$, representing the smallest and largest timestamps within its corresponding subtree. These time intervals are disjoint among all nodes in the same layer: for any two children of a parent node, all points in the left child expire before the parent node's timestamp, while all points in the right child enter after it. As a result, each endpoint of the query interval, t_l or t_r , falls within the time interval of exactly one node at each layer. In each layer, up to four nodes may be visited during reconstruction: the node containing t_l and its right sibling, and the node containing t_r and its left sibling. According to Lemma 1, Algorithm 7 visits $O(\log n)$ nodes in total when reconstructing the neighbor list for the given query interval, and according to lines 6-9 of Algorithm 7, number of points visited in each node cannot exceed M . Besides, because we terminate early when the candidate number exceeds M , the execution of *Select-Nbrs* requires only $O(M^2)$ time. In summary, the total time complexity for reconstructing the neighbor list for a point u over a query interval of arbitrary length is $O(M \log n)$. ■

Since the number of visited point is $O(\log N)$, the time complexity of supporting IANNS with compressed timestamp graph is $O(M \log^2 N)$, which is independent of the length of the query interval.

Then, we show that the time complexity of updating remain unchanged, which is still $O(\log^2 N)$. Specifically, we analyze the additional time required to precompute the neighbor list for large subtree. From the proof of Lemma 3, we know that

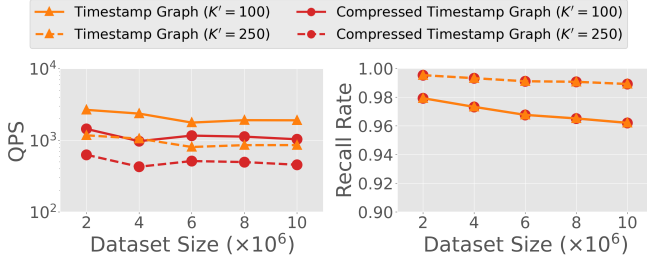


Fig. 8: Search performance of scalability test.

the total number of neighbors in the graph, i.e., $\sum_{i=1}^N n_i$ is $O(MN)$. Moreover, each neighborhood relation in the compressed timestamp graph can participate in the precomputation procedure for at most one expired point, implying that the amortized cost of precomputing neighbor list for a point is $\frac{M \times O(MN)}{N} = O(M^2)$. Since this cost is bounded within the time of the original update, the time complexity of updating compressed timestamp graph remains unchanged.

Space Complexity. Finally, we prove in Lemma 5 that the space complexity of compressed timestamp graph with pre-computed neighbor list of large subtree is still $O(MN)$.

Lemma 5: The space complexity of compressed timestamp graph with the precomputed neighbor list for subtree is $O(MN)$, where N is the size of dataset and M is the neighbor number for each point.

Proof: We begin by analyze the space for historic neighbor tree of a point with n total historic neighbors. According to the construction procedure of the historic neighbor tree, each leaf node of the tree contains a fixed number of μ points. Thus, for all layers above level $\lfloor \log \frac{n}{\mu M} \rfloor$, each node stores a pre-computed neighbor list of size M . The number of such upper-layer nodes is bounded by $2^{\lfloor \log \frac{n}{\mu M} \rfloor + 1} = O(\frac{n}{M})$, which implies that the total additional space required for storing the precomputed neighbor lists is $O(n)$. Therefore, the space complexity for each single historic neighbor tree remains unchanged. Since the total number of neighbor across all points is $\sum_{i=1}^N O(n_i)$, and the summation $\sum_{i=1}^N n_i$ is bounded by $2MN$, the total space complexity of the compressed timestamp graph with precomputed neighbor lists remains $O(MN)$. ■

B. Experiments on Scalability Test

In this part, we evaluate the scalability of the timestamp graph using the DEEP dataset [23]. Experiments are conducted with dataset sizes ranging from 2 million to 10 million vectors. Four metrics are used to assess performance, i.e., QPS, recall rate, update throughput, and memory usage.

Search Performance. Fig. 8 shows the recall rate and QPS of TANNs when varying dataset size. The neighbor number M is fixed at 16, with the search scope parameter k' set to 100 and 250 for the experiment. As the dataset size increases, the recall rate of the timestamp graph shows a slight decline for a fixed search scope. Besides, even with the largest dataset containing 10 million vectors, the timestamp graph achieves a QPS of 857 while maintaining a recall rate of 99%, showing good scalability of timestamp graph in search performance.

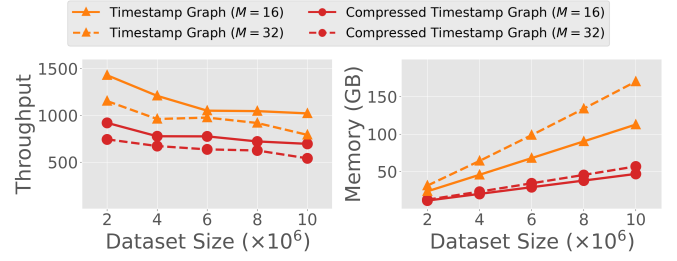


Fig. 9: Index construction in scalability test.

Index Construction Performance. Fig. 9 illustrates the update throughput and memory usage of timestamp graph as dataset size increases, with neighbor numbers set to 16 or 32. We observe that the update throughput decreases gradually as the dataset size grows. For instance, with $M = 16$, the update throughput of the compressed timestamp graph is 920, 777, 774, 721, 695 per second as the dataset size grows from 2 million to 10 million. Moreover, memory usage grows nearly linearly with the dataset size. The compressed timestamp graph requires 11.2GB of memory for 2 million vectors and 46.7GB for 10 million vectors. Besides, for a 10 million dataset with $M = 32$, compressed timestamp graph reduces memory usage by 66.8%, demonstrating the effectiveness of historic neighbor tree in optimizing the storage overhead.